

TEMA 3 – Firebase

Objetivo del tema

En este tema aprenderemos a conectar tu App con Firebase para realizar las operaciones básicas sobre una base de datos documental.

3.1 Firebase

Firestore es un recurso que amplía las posibilidades de las aplicaciones. Surgió en 2014 y es propiedad de Google. Ofrece diferentes servicios, desde autenticación a BBDD, servicio almacenamiento, hosting, configuración remota, etc. Es gratuita, aunque como siempre, ofrece una versión de pago.

Nosotros vamos a servirnos de FireBase como una Base de Datos On-Line. A diferencia de otras bases de datos, **no es SQL** sino orientada a **documentos**. Tienes más información sobre Firebase en la documentación de la asignatura de **ADT**. Nos basaremos en esa teoría y explicaciones para generar los ejemplos y apuntes necesarios para usar Firebase desde Kotlin en Android.

Vamos a trabajar bajo el supuesto de que tenemos ya definido un proyecto en Firebase:

<https://firebase.google.com/>

Nuestra Firestore Database es similar a esta:

🏠 > EmpresaBD > D2		
📁 (default)	📁 EmpresaBD	📁 D2
+ Iniciar colección	+ Agregar documento	+ Iniciar colección
EmpleadoBD	D1	+ Agregar campo
EmpresaBD >	D2 >	localizacion: "Bilbao"
		nombre: "Produccion"

Antes de hacer nada, vamos a **preparar un proyecto** de Android para que utilice Firestore Database. Por favor, mucho cuidado y paciencia que, si metes la pata, es difícil arreglarlo después y tendrás que empezar de cero.

Paso 1 – Google Services: Para que nuestra App pueda operar con ROOM, tiene que poder usar los **servicios de Google**. Añadimos los plugins al **build.gradle.kts (Proyecto)**:

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false

    // Google services
    alias(libs.plugins.google.services) apply false
}
```

A continuación, añades las siguientes entradas en el fichero **libs.versions.toml**:

```
[versions]
agp = "8.13.0"
kotlin = "2.2.20"
googleServices = "4.4.4"
```

Y al final:

```
[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
google-services = { id = "com.google.gms.google-services", version.ref = "googleServices" }
```

Al darle a sincronizar, el Android Studio descargará todas las dependencias que necesites de la versión que necesites, y podrás usarlas desde tu código.

En este documento tienes una versión resumida de los pasos para configurar el Gradle con ROOM. Dale un vistazo a los apuntes de Jetpack el documento **UD2.7 - Unique Activity**.

Paso 2 – Dependencias: Volvemos al **build.gradle.kts (App)** y le añadimos las dependencias de ROOM. Al principio del fichero ponemos:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)

    // Add the Google services Gradle plugin
    id("com.google.gms.google-services")
}
```

Y al final las dependencias:

```
// Import the Firebase BoM
implementation(platform( notation = "com.google.firebase:firebase-bom:33.0.0"))

// Firestore KTX (para Kotlin)
implementation("com.google.firebase:firebase-firestore-ktx")

// Now you can add the dependencies for Firebase products you want to use
// When using the BoM, don't specify versions in Firebase dependencies
//implementation(libs.firebase.analytics) // Optional
implementation("com.google.firebase:firebase-analytics") // Optional

}
```

En este caso, no he usado el fichero **libs.versions.toml** a modo ilustrativo, pero puedes utilizarlo si quieres. Le das a [sincronizar](#).

Importante

Por motivos que se me escapan, los de Firebase han dejado de sacar nuevas versiones de los módulos KTX, y han quitado las librerías KTX de Firebase Android BoM (v34.0.0).

Por tanto, si NO pones la 33.0.0, no tendrás ningún error de Gradle, pero en la MainActivity no te deja importar la librería "Firestore", por lo que si queremos hacer:

```
private val db = FirebaseFirestore.getInstance()
```

Nos va a dar un error de compilación.

¿Qué está mal? Nada en tu código ni en el Gradle. Simplemente, que no se descarga la librería "Firestore" porque la versión 34.0.0 o superior no la tiene. Entonces, ¿qué hay que usar entonces? Su API. Pero como el cambio lo han hecho en **Julio del 2025** hay que apañarse con una versión anterior para que todo funcione... en fin...

Paso 3 – Settings: Abrimos el fichero **settings.gradle.kts** y nos aseguramos de que los repositorios de Google y Maven Central están puestos. Deberías ver algo así:

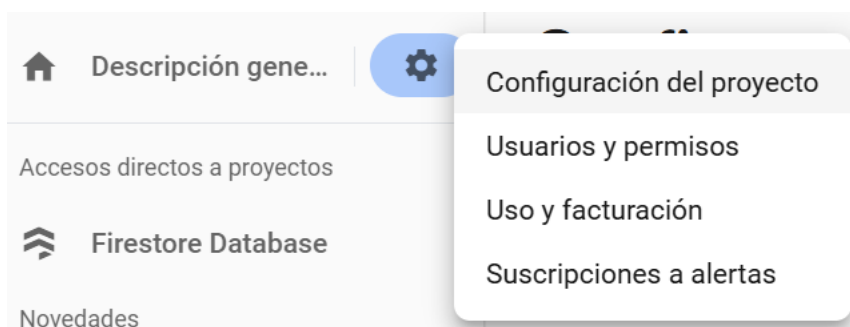
```
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex( groupRegex = "com\\.\\.android.*")
                includeGroupByRegex( groupRegex = "com\\.\\.google.*")
                includeGroupByRegex( groupRegex = "androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

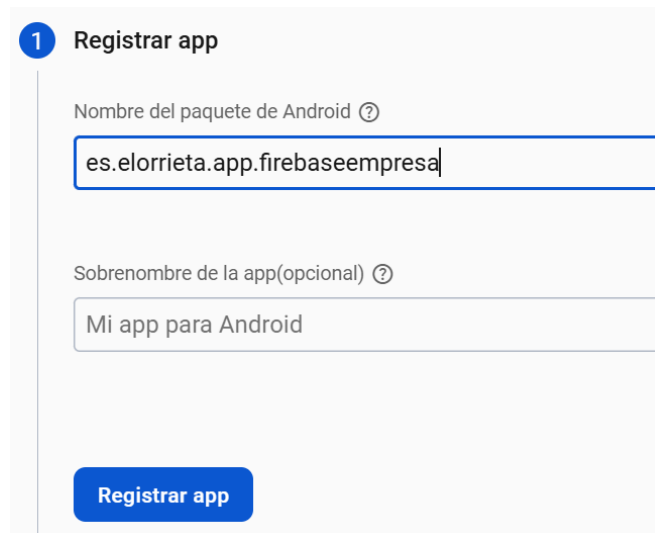
rootProject.name = "FireBaseEmpresas"
include( ...projectPaths = ":app")
```

Paso 4 – Registrar la App: Para poder acceder desde tu app a la Base de Datos es necesario primero **registrarla** en tu proyecto de Firebase. Esto nos genera un fichero json que añadiremos a nuestro proyecto Android, y que funcionará como identificador.

En la web de Firebase, vamos a **Configuración del Proyecto**, en la pestaña **General** bajamos hasta encontrar **Tus apps**. Allí le damos a **Agregar App**.

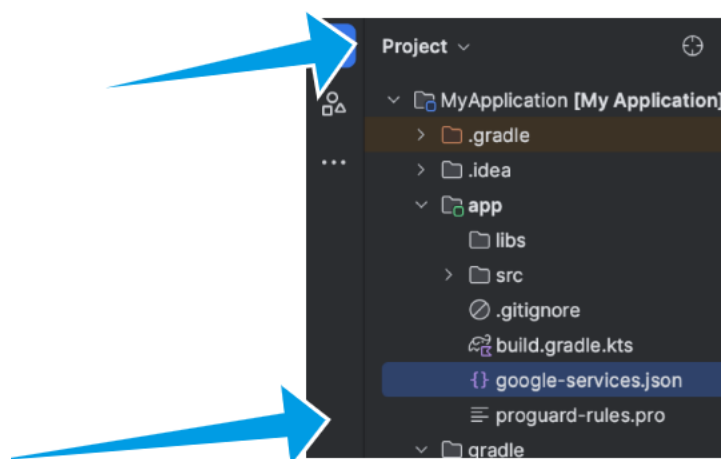


En la ventana emergente, le damos a Android y después indicaremos la ruta de paquetes de nuestra app de Android en la que se encuentra nuestra Actividad principal.



La siguiente ventana nos permitirá descargar un fichero **google-services.json** que deberemos incluir en nuestro proyecto Android. Verás que, al completar el registro, nos aparecerá una entrada en **Tus apps**.

El fichero **google-services.json** tiene que añadirse en una ruta específica: dentro de la carpeta app de tu proyecto, como ves en la imagen.



No te preocupes si lo añades, pero no lo ves en el Android Studio. Estar, está. Y si no lo tienes, te dará error.

Paso 4 – Permisos: Para que tu App tenga salida a Internet hay que darle permisos. A modo de recordatorio, vas al **AndroidManifest** y le añades:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

3.2 Consultas

Las consultas a una base de datos en FireBase son razonablemente sencillas. Tienes un ejemplo funcional junto a este documento, aquí solamente mencionaremos las partes importantes.

Paso 1 – Instanciar la BBDD: Para acceder a la base de datos, hay que instanciarla:

```
// Firestore Instance
private val db = FirebaseFirestore.getInstance()
```

Paso 2 – Evento: Vamos a añadir un evento a un botón para que nos cargue en un RecyclerView nos cargue todas las Empresas:

```
findViewById<Button>( id = R.id.buttonFindAllEnterprises).setOnClickListener {
    // A Mutable List in Kotlin is a list whose contents can be changed after it is created
    val enterpriseList = mutableListOf<Enterprise>()

    // Access to DB to retrieve "EmpresaBD" Documents. Note there are TWO possible events
    // for the call: onSuccess y onFailure. We must provide behaviour for both
    db.collection( collectionPath = "EmpresaBD")
        .get()
        .addOnSuccessListener { result ->
            // So, everything went OK. This DOES NOT MEAN we found any Documents
            // We loop and retrieve each Document individually
            // NOTE the Document ID must be assigned manually
            for (document in result) {
                val enterprise = document.toObject( valueType = Enterprise::class.java)
                enterprise.id = document.id // The ID must be assigned manually
                enterpriseList.add(enterprise)
                Log.d( tag = "Firestore", msg = "Enterprise: $enterprise")
            }
            // We update the adapter
            enterprisesAdapter.update( newEnterprises = enterpriseList)
        }
        .addOnFailureListener { exception ->

            // Something went wrong. This DOES NOT MEAN there was no Documents. This means
            // something serious, like server errors, etc.
            Toast.makeText(
                context = this,
                text = "Error when reading enterprises - $exception",
                duration = Toast.LENGTH_SHORT
            ).show()
            Log.e( tag = "Firestore", msg = "Error when reading enterprises", tr = exception)
        }
}
```

El código es extremadamente simple. Hacemos la petición con `db.collection("EmpresaBD")`. Esto nos va a devolver una serie de Documentos. Lo raro es cómo se tratan las respuestas de la Base de Datos: mediante **eventos**.

Si todo va bien, entonces se va a ejecutar la parte del **addOnSuccessListener**. Se recorren los resultados uno a uno, se convierten en objetos Enterprise, y se le pasan al adapter para cargar la tabla. Ten cuidado, que las ID (referencias internas de Firebase) se tienen que cargar explícitamente a mano.

```
for (document in result) {  
    val enterprise = document.toObject( valueType = Enterprise::class.java)  
    enterprise.id = document.id // The ID must be assigned manually  
    enterpriseList.add(enterprise)  
    Log.d( tag = "Firestore", msg = "Enterprise: $enterprise")  
}
```

Y recuerda: que no haya Documentos en la respuesta NO SIGNIFICA que haya algún error. Simplemente, que no hay Documentos. Es una respuesta válida.

Si algo ha pasado, entonces se va a ejecutar el **addOnFailureListener**. En este caso, nos limitaremos a informar del Error.

¿Te suena la estructura? Es muy similar a la forma en la que se tratan el retorno de parámetros de un Intent.

Paso 3 – El Documento: Cuando definamos la clase Enterprise que usaremos para gestionar los Documentos es muy importante inicializar TODOS sus atributos a sus valores por defecto. Firestore necesita constructores vacíos para serializar y deserializar correctamente las cosas, si no lo hacemos, no funcionará.

```
// Los valores por defecto (= "", = 0) son necesarios porque Firestore  
// necesita un constructor vacío para deserializar.  
data class Enterprise(var id: String = "", 6 Usages  
    var nombre: String = "",  
    var localizacion: String = "")
```

3.3 Inserciones

Cuando insertamos un documento en Firebase, podemos hacerlo de dos formas:

- Dejando que Firebase autogenera la ID
- Dándole la ID nosotros.

Para darle nosotros la ID, basta con hacer:

```
val enterprise = Enterprise( id = "D3", nombre = "Softcom", localizacion = "Vitoria")
db.collection( collectionPath = "EmpresaBD")
    .document( documentPath = enterprise.id)
    .set(enterprise)
    .addOnSuccessListener { result ->
        Log.d( tag = "Firestore", msg = "Enterprise added: $enterprise")
    }
    .addOnFailureListener { exception ->
        Log.e( tag = "Firestore", msg = "Error when inserting enterprises", tr = exception)
    }
```

En este ejemplo extraemos la id del POJO Enterprise. Esto es por simplicidad, pero en realidad provoca que el Documento en Firebase aparezca con un campo extra ID. Esto se debe a que, al fin y al cabo, Enterprise también tiene un atributo ID.

Por tanto, recuerda que las ID de Firebase deberían de tratarse por separado del resto de los campos del Documento.

Para dejar la autogeneración, basta con hacer:

```
val enterprise = Enterprise( id = "D3", nombre = "Softcom", localizacion = "Vitoria")
db.collection( collectionPath = "EmpresaBD")
    .add(enterprise)
    .addOnSuccessListener { result ->
        Log.d( tag = "Firestore", msg = "Enterprise added: $enterprise")
    }
    .addOnFailureListener { exception ->
        Log.e( tag = "Firestore", msg = "Error when inserting enterprises", tr = exception)
    }
```


3.4 Borrado

Cuando eliminamos un documento en Firebase, podemos hacerlo de dos formas:

- Eliminándolo directamente por su ID
- Realizando una búsqueda del Documento por otro criterio, y borrándolo entonces.

Para hacerlo por ID, basta con hacer:

```
val id = "D3"

db.collection( collectionPath = "EmpresaBD")
  .document( documentPath = id)
  .delete()
  .addOnSuccessListener { result ->
    Log.d( tag = "Firestore", msg = "Enterprise deleted, ID: $id")
  }
  .addOnFailureListener { exception ->
    Log.e( tag = "Firestore", msg = "Error when inserting enterprises", tr = exception)
  }
```

Mientras que para el borrado por otro criterio, hacemos primero una búsqueda:

```
val name = "Softcom"

db.collection( collectionPath = "EmpresaBD")
  .whereEqualTo( field = "nombre", value = name)
  .get()
  .addOnSuccessListener { querySnapshot ->
    for (document in querySnapshot.documents) {
      document.reference.delete()
      .addOnSuccessListener {
        Log.d( tag = "Firestore", msg = "Enterprise deleted: $name")
      }.addOnFailureListener { exception ->
        Log.e( tag = "Firestore", msg = "Error when inserting enterprises", tr = exception)
      }
    }
  }
  .addOnFailureListener { exception ->
    Log.e( tag = "Firestore", msg = "Error when inserting enterprises", tr = exception)
  }
```

Recuerda: un delete () ordena un borrado, pero no hay forma de saber si ha borrado o no. Es por esto que primero se hace una búsqueda del documento a borrar y si existe, se borra.

Ejercicio 24

Crea una App que sea capaz de almacenar tus canciones favoritas en Firebase. Se guardará el título de la canción, el autor, y la URL de YouTube donde se puede escuchar. La App debe permitir listar todas las canciones, añadir una canción, borrar una canción y modificar sus datos. Las canciones deberán aparecer en formato lista (RecyclerView).

Si seleccionas un elemento de la lista, se realizará un Intent a YouTube y se reproducirá la canción.